Georgia Institute of Technology

CS 4365/6365: IEC

Spring 2026

Project Checkpoint Report

Group: 38
Name(s): Ryan Pham
Project: Automated Vendor SLA Tracker

Project Plan (Plan):

Point A

- **Background:** Modern enterprise operations rely on third party vendors for logistics, shipping, and supply chain fulfillment. These vendors are legally bound to accept delivery timeframes and uphold performance thresholds according to the Service Level Agreements (SLAs) between the enterprise and vendor. To ensure business efficiency, operational planning, and cost control, enterprises must ensure these metrics are tracked accurately. However, companies typically rely on reactive methods, such as static, end of month database reporting, to identify when a vendor breaches their contract. Because delays are caught weeks after they happen, financial losses and operational bottlenecks can't be undone.
- **Concept:** We propose an "Automated Vendor SLA Tracker," an event driven Extract, Transform, Load (ETL) pipeline and microservice. Instead of waiting for monthly reports, the concept is to continuously ingest unstructured, live JSON data, standardize it into a dashboard for performance metrics, and trigger automated, real time alerts when delays occur. To simulate the live data stream of an enterprise vendor, this project will utilize the *MBTA* public transit API as a proxy to track real time feed.
- **Related Work:**
    - Current solutions used by large enterprises often involve

Electronic Data Interchange (EDI) or batch files to update their data once a day using proprietary software suites.
    - Standard Business intelligence has been following a passive model leveraging ETL into tools like Tableau and PowerBI. These tools require analysts to log in, refresh the data, and spot the SLA breaches leading to “dashboard fatigue”
    - *MBTA* API has mainly been utilized by applications such as Google Maps and Apple Transit to display *MBTA* vehicle locations and times for commuters.
- **Starting Point:** The starting point for this project is to build the foundational Extract, Transform, Load pipeline. This initial phase will focus entirely on establishing the connection to the *MBTA* API, parsing the live JSON files, and loading the metrics into a database for future analysis.
    - API Ingestion: Establish a reliable connection to the *MBTA* API using Python, handle API key authentication, and successfully downloading the raw JSON payload
    - Data Parsing & Validation: Create parsing logic to track specific data fields for SLA tracking, including error-handling for missing data or API dropouts.
    - Storage Architecture: Design a relational schema with tables that can be queried for future dashboard visualization.

Point B: Present the deliverables planned for the project at the end of the semester. • Milestone chart: A weekly chart showing the project plan and the planned work for each teammate.

Point B

By the end of the semester, this project will deliver a fully functional, event-driven analytics system comprising the following tangible components:

- ❖ Automated ETL Pipeline (Python): An automated script that authenticates with the *MBTA* API, extracts the live JSON vehicle data, handles missing or out of order data payloads, calculates schedule deviations, and loads the metrics into the database

- Relational Database Architecture: A fully implemented database that stores vehicle status recordings and logs calculated SLA breaches
- Event-Driven Alerting Microservice: A python microservice that monitors incoming data and automatically fires a webhook alerts to a communication channel the moment a vehicle exceeds the delay threshold. It will include state tracking and alert throttling to prevent constant, noisy alerts when data briefly fluctuates.
- Business Intelligence Dashboard: An interactive dashboard connected directly to the database that visualizes KPI metrics live.
- System Architecture Documentation: A GitHub repository containing the complete code, a system architecture diagram, and deployment instructions for pipeline and microservice interactions.

Milestone Chart:

Week Number	Deliverables
3 (Ckpt 0)	Reproducibility Assignment
6 (Ckpt 1)	Finalize Project Proposal
7	Establish API Ingestion + Prototype Ingestion Pipeline Slice
8 (Ckpt 2)	Database Architecture: Relational Schema + Load Logic
9	Develop Transformation Logic and Performance Metric Calculation
10 (Ckpt 3)	Alerting Microservice
11	Dashboard Integration
12 (Ckpt 4)	Finalize Dashboard UI and implement advanced Alerting Logic
13	Bug Tests, Overload Testing, and Alert Accuracy
14	GitHub Documentation and Code Cleanup
15 (Ckpt 5)	Project Completion, Final Submission, Demo

Current progress report (Match):

- **Work Done:** Over the past two weeks, I successfully finalized the transformation logic and built the event-driven alerting microservice. I rewrote the “Transform” phase in Python to parse the complex JSONAPI structures provided by the Boston MBTA API. This included deduplicating records to only track the next immediate stop, utilizing datetime math according to the UTC timezone to calculate exact ETAs, and calculating a live SLA Compliance Percentage. For the alerting microservice, I built a background daemon script that constantly scans the SQLite database for new SLA breaches. It successfully formats and pushes a live alert to an enterprise Slack channel via a secure Webhook. Using a second database table, I implemented a 15 minute cooldown time to prevent the chat from being spammed with alerts.
- **Milestone Chart Comparison:** I am currently caught up with the milestone chart. While the deliverables were completed two days behind the initial checkpoint schedule, the requirements for Week 9 (Develop Transformation Logic and Performance Metric Calculation) and Week 10 (Alerting Microservice) are now fully functional and met.
- **Immediate Planned Work:** The immediate goal for Week 11 is “Dashboard Integration.” I will connect the populated SQLite database to a business intelligence or visualization tool to create an interactive dashboard. This will allow for live filtering of the KPI metrics. Then I will ensure the Dashboard UI is well made and easily navigable for Week 12.
- **Changes to the Original Plan:** The original plan was to utilize the MARTA API to simulate the live data stream. However, due to severe and persistent reliability issues with the MARTA server dropping connections, I pivoted to the more reliable Boston MBTA API. This required a complete rewrite of the JSON parsing logic.

Supporting Evidence (Factual):

- GitHub Repository: https://github.com/ryandude99/Automated-Vendor-SLA-Tracker
- Checkpoint 3: All deliverables for Weeks 9 and 10 are pushed to the main branch and are fully verifiable:
    - api_ingestion.py: Demonstrates the completed transformation logic, including the successful pivot to the MBTA JSONAPI and the implementation of datetime math to calculate exact ETAs and performance metrics.
    - alertService.py: Contains the fully functional alerting microservice. This file demonstrates completeness of the Slack Webhook construction, the use of the alertState database to create a 15 minute cooldown between alerts, and a daemon loop to consistently check for new breaches to alert.

y M | alertService.py U | databaseSetup.py | slaTracker.db M

Rows: 17 | Filter 17 rows

	logID	trainID	station	minutes...	isBreach	timestamp
1	1	R-5488A785	70066	0	☐	2026-03-26 02:...
2	2	R-5488AD41	70084	0	☐	2026-03-26 02:...
3	3	R-5488AED9	70063	0	☐	2026-03-26 02:...
4	4	R-5488A7FB	70067	0	☐	2026-03-26 02:...
5	5	R-5488B00E	70078	0	☐	2026-03-26 02:...
6	6	R-5488AD4B	70075	0	☐	2026-03-26 02:...
7	7	R-5488AD94	70081	0	☐	2026-03-26 02:...
8	8	R-54889C01	70096	0	☐	2026-03-26 02:...
9	9	R-5488AD46	70093	0	☐	2026-03-26 02:...
10	10	R-5488ADEA	70095	0	☐	2026-03-26 02:...
11	11	R-5488A4B7	70082	1	☐	2026-03-26 02:...
12	12	R-54887D01	70101	1	☐	2026-03-26 02:...
13	13	R-5488AF9E	70104	1	☐	2026-03-26 02:...
14	14	R-5488AD7C	70087	3	☐	2026-03-26 02:...
15	15	R-5488AD45	70088	3	☐	2026-03-26 02:...
16	16	R-5488AD40	70063	4	☐	2026-03-26 02:...
17	17	R-5488A7FF	70104	11	☑	2026-03-26 02:...

Figure 1: Successful data ingestion and relational storage within the trainSlaLogs SQLite database table.

```
ers/drybo/OneDrive/Desktop/Automated Vendor SLA Tracker/api_ingestion.py"
Fetching live data from MBTA...
Successfully pulled 421 predictions.

Total Trains: 17
Total On Time: 15
Total Breaches: 2
SLA Compliance: 88.2%
```

Figure 2: Execution logs from the transformation engine demonstrating successful data deduplication and system-wide SLA performance calculations.

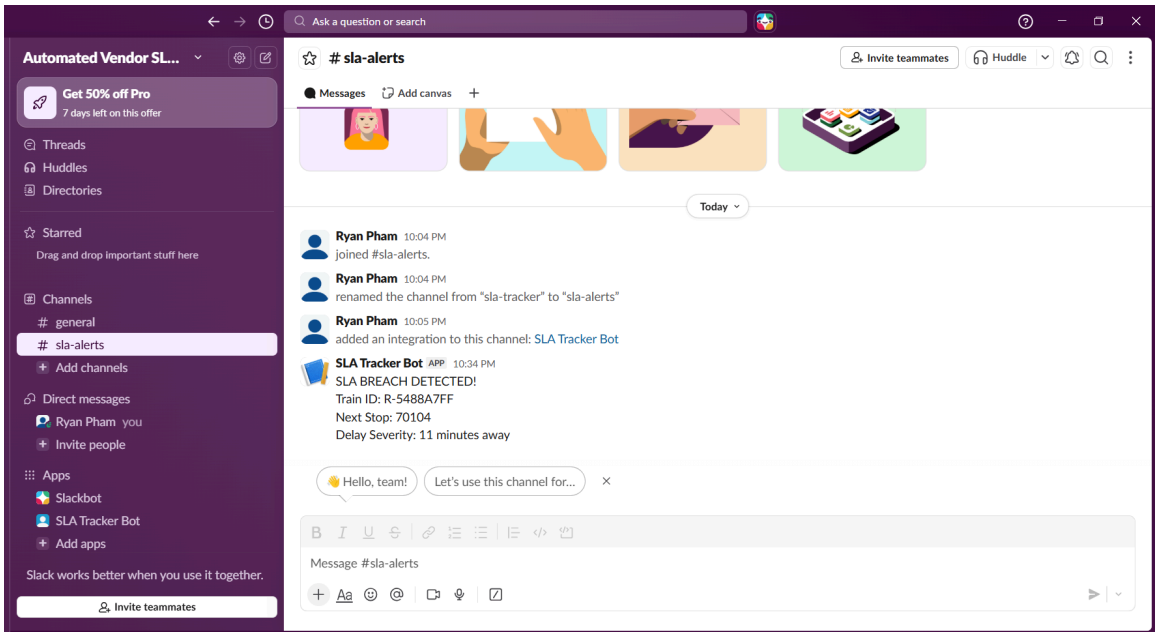

Figure 3: Real-time SLA breach notification successfully formatted and delivered to an enterprise Slack channel via the event-driven alerting microservice.

Skill Learning Report:

- Advanced API Integration and Datetime Mathematics
  - Progress: I learned how to handle architectural pivots by successfully migrating my ingestion engine from the MARTA API to the MBTA JSONAPI. This required dictionary unwrapping to navigate nested payloads. Additionally, I learned how to implement timezone aware datetime mathematics to dynamically calculate live train ETAs. I also ensured debugging logic for stale and negative timestamps.

- Event Driven Microservices and State Tracking
    - Progress: I developed a continuous event driven background daemon. I learned how to implement state tracking using a SQLite table. This taught me how to logically throttle system events to specifically enforce a cooldown to prevent duplicate alerts from spamming the slack channel regarding the same vehicle.
- Enterprise Integrations and Secret Management
    - I gained experience integrating external corporate communication tools by formatting and pushing JSON payload to a Slack Webhook. I also learned critical software security practices after triggering a strict GitHub Push Protection error. I successfully separated sensitive Slack credentials by implementing a .env file, .gitignore file, and the python-dotenv library.

Self-Evaluation:

- **Scope (110%):** The scope of this project merits a 110% rating because it goes beyond a standard, static data analysis assignment. It implements a live, event driven ETL pipeline and requires actively handling unpredictable API data formats.
- **Match (100%):** I deducted 20% from the base score because the Week 9 and Week 10 deliverables were completed two days past the initial checkpoint timeline. However, I added 10% back to the score because I exceeded expectations by already completing the Advanced Alerting Logic designed for Week 12. Additionally, I added 10% back since I had to rewrite code to architecturally pivot from the MARTA API to the MBTA JSONAPI
- **Factual (100%):** The factual score for this project is exactly 100% because my entire pipeline is fully verifiable on GitHub. The repository clearly documents my architectural pivot to the MBTA API, Figure 2 shows the Week 9 transformation and performance metric calculations, and Figure 3 shows the completed Week 10 alerting microservice.

AI Evaluation (Copied Directly from ChatGPT):

- **Scope (108%):** The scope of this project remains well above expectations for the course. At this stage, the project is no longer just a planned architecture but now includes a fully implemented transformation engine and an event-driven alerting microservice integrated with an external platform. The addition of real-time Slack webhook alerts, cooldown-based state tracking, and dynamic SLA metric calculations demonstrates strong enterprise-level thinking. The system reflects a complete pipeline from ingestion to actionable insight, which aligns closely with real-world vendor monitoring systems. While the project is highly developed, some aspects such as deployment environment, scalability beyond SQLite, and dashboard implementation are still in progress, which slightly limits the scope from reaching the highest possible score.
- **Match (98%):** The match between the planned milestones and completed work is very strong. The deliverables for both Week 9 (Transformation Logic and Performance Metric Calculation) and Week 10 (Alerting Microservice) have been completed and are fully functional. The report clearly explains the implementation details, including JSON parsing, datetime calculations, and alert triggering logic. Although the work was completed slightly behind schedule by two days, the project has effectively caught back up and is now aligned with the milestone chart. Additionally, completing portions of advanced alerting logic earlier than planned demonstrates progress that slightly exceeds expectations, but the small delay prevents a full score above 100%.
- **Factual (95%):** The project provides strong supporting evidence through a GitHub repository and references to specific files such as the ingestion/transformation script and the alerting microservice. The included figures showing database storage, transformation logs, and Slack alert outputs further support the claims made in the report. However, the factual score is slightly reduced because the evidence could be more directly accessible to reviewers. Providing direct file links, short execution instructions, or small inline data samples would make verification faster and more efficient. Despite this, the implementation is clearly real and well-documented, supporting a high factual score.